

Python - List Comprehension

List Comprehension in Python

A **list comprehension** is a concise way to create lists. It is similar to set builder notation in mathematics. It is used to define a **list** based on an existing iterable object, such as a list, tuple, or string, and apply an expression to each element in the iterable.

Syntax of Python List Comprehension

The basic syntax of list comprehension is –

```
new_list = [expression for item in iterable if condition]
```

Where,

- **expression** is the operation or transformation to apply to each item in the iterable.
- **item** is the variable representing each element in the iterable.
- **iterable** is the sequence of elements to iterate over.
- **condition (optional)** is an expression that filters elements based on a specified condition.

Example of Python List Comprehension

Suppose we want to convert all the letters in the string "hello world" to their upper-case form. Using list comprehension, we iterate through each character, check if it is a letter, and if so, convert it to uppercase, resulting in a list of uppercase letters –

```
string = "hello world"
uppercase_letters = [char.upper() for char in string if char.isalpha()]
print(uppercase_letters)
```

The result obtained is displayed as follows –

```
['H', 'E', 'L', 'L', 'O', 'W', 'O', 'R', 'L', 'D']
```

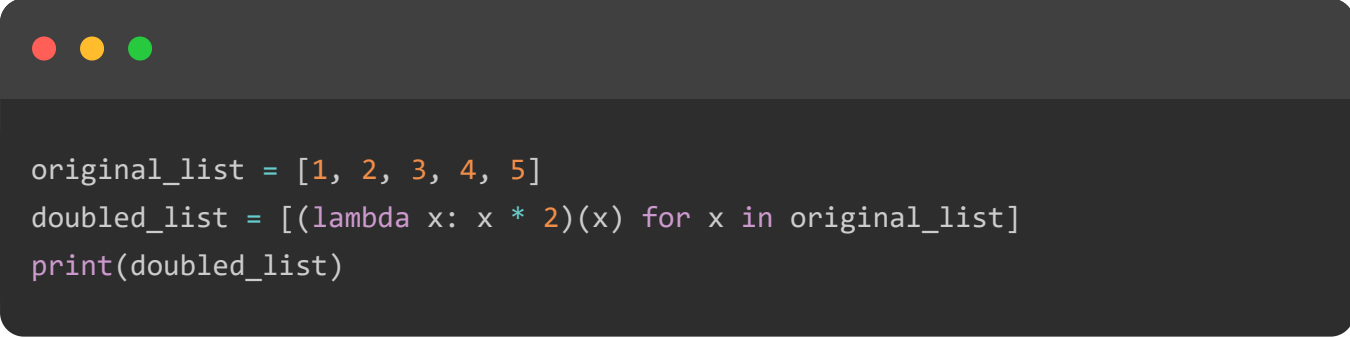
List Comprehensions and Lambda

In Python, **lambda** is a keyword used to create anonymous functions. An anonymous function is a function defined without a name. These functions are created using the **lambda** keyword followed by a comma-separated list of arguments, followed by a colon :, and then the expression to be evaluated.

We can use list comprehension with lambda by applying the lambda function to each element of an iterable within the comprehension, generating a new list.

Example

In the following example, we are using list comprehension with a lambda function to double each element in a given list "original_list". We iterate over each element in the "original_list" and apply the lambda function to double it –



```
original_list = [1, 2, 3, 4, 5]
doubled_list = [(lambda x: x * 2)(x) for x in original_list]
print(doubled_list)
```

Following is the output of the above code –

```
[2, 4, 6, 8, 10]
```

Nested Loops in Python List Comprehension

A nested loop in Python is a loop inside another loop, where the inner loop is executed multiple times for each iteration of the outer loop.

We can use nested loops in list comprehension by placing one loop inside another, allowing for concise creation of lists from multiple iterations.

Example

In this example, all combinations of items from two lists in the form of a tuple are added in a third list object –

```
list1=[1,2,3]
list2=[4,5,6]
CombLst=[(x,y) for x in list1 for y in list2]
print (CombLst)
```

It will produce the following output –

```
[(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)]
```

Conditionals in Python List Comprehension

Conditionals in Python refer to the use of statements like "if", "elif", and "else" to control the flow of a code based on certain conditions. They allow you to execute different blocks of code depending on whether a condition evaluates to "True" or "False".

We can use conditionals in list comprehension by including them after the iterable and before the loop, which filters elements from the iterable based on the specified condition while generating the list.

Example

The following example uses conditionals within a list comprehension to generate a list of even numbers from 1 to 20 –

```
list1=[x for x in range(1,21) if x%2==0]
print (list1)
```

We get the output as follows –

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

List Comprehensions vs For Loop

List comprehensions and for loops are both used for iteration, but they differ in terms of syntax and usage.

List comprehensions are like shortcuts for creating lists in Python. They let you generate a new list by applying an operation to each item in an existing list.

For loop, on the other hand, is a control flow statement used to iterate over elements of an iterable one by one, executing a block of code for each element.

List comprehensions are often preferred for simpler operations, while for loops offer more flexibility for complex tasks.

Example Using For Loop

Suppose we want to separate each letter in a string and put all non-vowel letters in a list object. We can do it by a for loop as shown below –

```
chars=[]
for ch in 'TutorialsPoint':
    if ch not in 'aeiou':
        chars.append(ch)
print (chars)
```

The chars list object is displayed as follows –

```
['T', 't', 'r', 'l', 's', 'P', 'n', 't']
```

Example Using List Comprehension

We can easily get the same result by a list comprehension technique. A general usage of list comprehension is as follows –

```
listObj = [x for x in iterable]
```

Applying this, chars list can be constructed by the following statement –



```
chars = [ char for char in 'TutorialsPoint' if char not in 'aeiou']  
print (chars)
```

The chars list will be displayed as before –

```
['T', 't', 'r', 'l', 's', 'P', 'n', 't']
```

Example

The following example uses list comprehension to build a list of squares of numbers between 1 to 10 –



```
squares = [x*x for x in range(1,11)]  
print (squares)
```

The squares list object is –

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Advantages of List Comprehension

Following are the advantages of using list comprehension –

- **Conciseness** – List comprehensions are more concise and readable compared to traditional for loops, allowing you to create lists with less code.
- **Efficiency** – List comprehensions are generally faster and more efficient than for loops because they are optimized internally by Python's interpreter.
- **Clarity** – List comprehensions result in clearer and more expressive code, making it easier to understand the purpose and logic of the operation being

performed.

- **Reduced Chance of Errors** – Since list comprehensions are more compact, there is less chance of errors compared to traditional for loops, reducing the likelihood of bugs in your code.

TOP TUTORIALS

[Python Tutorial](#)

[Java Tutorial](#)

[C++ Tutorial](#)

[C Programming Tutorial](#)

[C# Tutorial](#)

[PHP Tutorial](#)

[R Tutorial](#)

[HTML Tutorial](#)

[CSS Tutorial](#)

[JavaScript Tutorial](#)

[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)

[Amazon Web Services Tutorial](#)

[Microsoft Azure Tutorial](#)

[Git Tutorial](#)

[Ethical Hacking Tutorial](#)

[Docker Tutorial](#)

[Kubernetes Tutorial](#)

[DSA Tutorial](#)

[C++ Best Tutorial](#)



Chapters ▾

Categories ≡

[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)

[Java & Spring Boot Advanced Certification](#)

Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.